



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y
Tecnología

Grado en Ingeniería en Ciencia de Datos e Inteligencia Artificial
**Prototipo de negociación automatizada en cadena de suministro mediante agentes basados en
LLMs**

Trabajo fin de estudio presentado por:	Santiago Martín Cabrera
Tipo de trabajo:	Tipo 2. Desarrollo software
Director/a:	Javier Escobar Ortiz
Fecha:	30/04/2026

Resumen

Esta segunda entrega presenta el avance realizado en el Trabajo Fin de Grado, centrado en el diseño e implementación de un prototipo software para negociación automatizada en un contexto de cadena de suministro. La finalidad principal del trabajo es analizar, en un entorno controlado, si un modelo de lenguaje de gran tamaño puede actuar como agente negociador en una operación de compra-venta bajo un contexto inicial, unos objetivos privados y unos parámetros explícitos. El sistema se plantea como una simulación entre dos agentes, comprador y vendedor, que intercambian propuestas sobre precio unitario, cantidad y plazo de entrega. A partir del feedback recibido tras la primera entrega, el trabajo se ha reorientado para evitar una confianza directa e incontrolada en modelos de lenguaje. Por ello, el prototipo incorpora un protocolo explícito de negociación, validación estructurada de acciones, restricciones públicas del escenario, guardrails privados por agente, trazabilidad de cada turno y métricas de evaluación. Además, se ha integrado un proveedor local basado en Ollama, manteniendo un proveedor determinista como línea base. Los resultados preliminares muestran que el sistema puede ejecutar negociaciones, registrar el proceso y bloquear salidas inválidas generadas por el proveedor LLM. Esta entrega no pretende demostrar que los LLMs estén preparados para sustituir a vendedores o teleoperadores comerciales en entornos reales, sino estudiar su viabilidad preliminar como componentes de apoyo en futuros procesos comerciales automatizados.

Palabras clave: cadena de suministro; negociación automatizada; sistemas multiagente; LLMs; guardrails.

Índice de contenidos

1. Introducción

1.1. Motivación

1.2. Planteamiento del trabajo

1.3. Estructura del trabajo

2. Contexto y estado del arte

2.1. Contexto del problema

2.2. Estado del arte

2.3. Conclusiones

3. Objetivos concretos y metodología de trabajo

3.1. Objetivo general

3.2. Objetivos específicos

3.3. Metodología del trabajo

4. Desarrollo específico de la contribución

4.1. Alcance funcional del prototipo

4.2. Arquitectura software

4.3. Protocolo de negociación

4.4. Guardrails, validación y trazabilidad

4.5. Integración local con LLM mediante Ollama

5. Código fuente y datos analizados

5.1. Repositorio y organización del código

5.2. Escenarios simulados

5.3. Pruebas y resultados preliminares

6. Conclusiones y trabajo futuro

6.1. Conclusiones provisionales

6.2. Limitaciones y prospectiva

Referencias bibliográficas

Anexos

Índice de figuras

No se incluyen figuras en esta segunda entrega. La arquitectura se documenta mediante tablas y descripción textual.

Índice de tablas

Tabla 1. Variables negociadas en el prototipo.

Tabla 2. Componentes principales del prototipo.

Tabla 3. Acciones del protocolo de negociación.

Tabla 4. Métricas previstas para la evaluación.

Tabla 5. Pruebas preliminares realizadas.

1. INTRODUCCIÓN

1.1.MOTIVACIÓN

La gestión de la cadena de suministro implica coordinar decisiones entre organizaciones con intereses parcialmente alineados, pero no necesariamente idénticos. En una relación de compra-venta, el comprador suele buscar precios reducidos, cantidades ajustadas a su necesidad y plazos de entrega tempranos, mientras que el vendedor persigue precios suficientes, cantidades rentables y plazos compatibles con su capacidad operativa. Esta tensión convierte la negociación en un proceso crítico para el coste, la disponibilidad y la eficiencia de la cadena de suministro.

Los sistemas multiagente y la negociación automatizada han sido estudiados desde hace décadas como mecanismos para apoyar procesos de decisión distribuidos. La literatura clásica ya señalaba el potencial de los agentes software para representar preferencias, intercambiar ofertas y apoyar transacciones en entornos digitales (Guttman et al., 1998; Jennings et al., 2001). Más recientemente, los modelos de lenguaje de gran tamaño han abierto nuevas posibilidades para construir agentes capaces de interpretar instrucciones, razonar sobre objetivos y generar respuestas en lenguaje natural o en formatos estructurados (Guo et al., 2024; Wang et al., 2024).

No obstante, el uso de LLMs en decisiones empresariales plantea riesgos evidentes. Un agente basado en un modelo de lenguaje puede generar respuestas inválidas, aceptar condiciones poco convenientes, incumplir restricciones o ser influido por el contexto de la conversación. Por ello, este TFE no plantea delegar de forma directa el beneficio de una empresa en un LLM, sino estudiar cómo puede integrarse un agente conversacional dentro de un entorno controlado, con reglas explícitas, validación estructurada y mecanismos de seguridad.

A partir de esta motivación, el trabajo se orienta al desarrollo de un prototipo software evaluable. La finalidad no es construir un sistema listo para uso empresarial real, sino analizar si un agente basado en LLM, sometido a contexto inicial, objetivos privados y restricciones explícitas, puede participar de forma controlada en una negociación de compra-venta. Esta orientación permite estudiar la posible viabilidad futura de agentes comerciales automatizados, sin delegar de forma directa la decisión empresarial en el modelo.

1.2.PLANTEAMIENTO DEL TRABAJO

El trabajo consiste en diseñar e implementar un prototipo en Python en el que dos agentes representan a un comprador y a un vendedor dentro de una negociación bilateral de cadena de suministro. La negociación se limita, de forma deliberada, a tres variables: precio unitario, cantidad y plazo de entrega. Esta acotación permite mantener un alcance razonable para un TFE de tipo desarrollo software y, al mismo tiempo, conservar una relación clara con problemas reales de aprovisionamiento.

Tras el feedback recibido en la primera entrega, se ha concretado el significado de “alcanzar un acuerdo”. En este prototipo, un acuerdo válido no se infiere automáticamente por coincidencia o solapamiento entre ofertas. Solo se considera que existe acuerdo cuando un agente acepta explícitamente una propuesta válida de la contraparte, dicha propuesta no ha sido rechazada previamente, respeta las restricciones públicas del escenario y cumple los guardrails privados del agente que acepta.

La solución combina un motor de negociación determinista con proveedores de acciones. El proveedor puede ser un mock determinista, utilizado como línea base, o un proveedor local basado en Ollama y un modelo LLM ejecutado en la máquina del autor. En ambos casos, el proveedor únicamente propone una acción candidata; la autoridad final sobre validez, restricciones, guardrails y cierre de acuerdos permanece en el motor y el validador del sistema.

1.3. ESTRUCTURA DEL TRABAJO

Esta segunda entrega se organiza en seis capítulos. El primer capítulo introduce la motivación y el planteamiento actualizado del trabajo. El segundo revisa el contexto y el estado del arte inicial. El tercer capítulo concreta los objetivos y la metodología de desarrollo. El cuarto capítulo describe el avance específico del prototipo software. El quinto presenta la organización del código, los escenarios simulados y las pruebas preliminares realizadas. Finalmente, el sexto capítulo recoge conclusiones provisionales, limitaciones y líneas de trabajo pendientes.

2. CONTEXTO Y ESTADO DEL ARTE

2.1.CONTEXTO DEL PROBLEMA

La negociación en cadena de suministro es un proceso en el que las partes intentan alcanzar condiciones aceptables en torno a variables que afectan directamente a costes, servicio y capacidad operativa. En el caso analizado, el comprador y el vendedor negocian sobre tres dimensiones básicas: precio unitario, cantidad y plazo de entrega. Aunque el escenario es simplificado, estas variables permiten representar tensiones habituales: menor precio frente a margen, mayor cantidad frente a capacidad, y entrega temprana frente a restricciones logísticas.

El interés del trabajo no reside en sustituir a negociadores humanos ni en desplegar una herramienta en producción, sino en estudiar cómo se puede construir un entorno experimental donde agentes basados en LLMs participen en una negociación comercial sin perder el control del proceso. Este matiz es fundamental: el prototipo no concede al modelo libertad absoluta, sino que encapsula sus respuestas en acciones estructuradas y aplica reglas de validación antes de aceptar cualquier resultado. Desde esta perspectiva, el TFE analiza una pregunta de fondo: hasta qué punto un LLM puede comportarse como agente de apoyo en una negociación de compra-venta cuando opera dentro de límites técnicos y empresariales definidos.

El contexto se delimita a una negociación bilateral entre un comprador y un vendedor. Esta decisión reduce la complejidad del problema y permite observar de forma más clara la evolución de ofertas, contraofertas, rechazos, abandonos y aceptaciones. En fases posteriores podrían añadirse más proveedores, criterios multicriterio o datos reales, pero para esta etapa el foco se mantiene en el diseño de un núcleo robusto y defendible.

2.2.ESTADO DEL ARTE

El estado del arte del trabajo se apoya en cuatro líneas principales. La primera es la negociación automatizada, que estudia cómo agentes software pueden representar intereses, intercambiar propuestas y llegar a acuerdos bajo restricciones. Trabajos clásicos como los de Guttman et al. (1998) y Jennings et al. (2001) proporcionan una base conceptual sobre comercio mediado por agentes, protocolos de negociación y problemas asociados a la automatización de decisiones.

La segunda línea corresponde al uso de agentes en cadena de suministro. En este ámbito, se han propuesto enfoques que utilizan agentes para apoyar negociaciones entre organizaciones, incorporar conocimiento del dominio y organizar procesos de aprovisionamiento. Fang y Wong (2010) aplican razonamiento basado en casos en negociaciones de cadena de suministro, mientras que Wang et al. (2013) proponen un enfoque ontológico para organizar negociaciones asistidas por sistemas multiagente.

La tercera línea se relaciona con los agentes basados en LLMs. Las revisiones recientes de Guo et al. (2024) y Wang et al. (2024) muestran que los modelos de lenguaje permiten construir agentes capaces de razonar, planificar, interactuar con herramientas y operar en entornos complejos. Sin embargo, estos trabajos también evidencian limitaciones relevantes, como alucinaciones, errores de seguimiento de instrucciones, fragilidad ante contextos largos y dificultades de evaluación.

La cuarta línea se centra en la evaluación de capacidades de negociación en modelos de lenguaje. Davidson et al. (2024) y Bianchi et al. (2024) analizan el comportamiento de modelos en juegos o entornos de negociación, lo que resulta especialmente relevante para este TFE. Estos trabajos refuerzan la idea de que la negociación puede utilizarse como escenario de evaluación de agencia, pero también muestran la necesidad de medir resultados y no basarse únicamente en la apariencia conversacional de las respuestas.

En conjunto, la revisión inicial sugiere una oportunidad clara: combinar la estructura de la negociación automatizada clásica con la flexibilidad lingüística de los LLMs, pero sin abandonar la validación formal. Por ello, el prototipo desarrollado se centra en acciones estructuradas, guardrails, trazabilidad y métricas de evaluación.

2.3.CONCLUSIONES

La revisión del contexto y del estado del arte confirma que el trabajo debe evitar un planteamiento excesivamente teórico o puramente conversacional. Para que el TFE sea defendible, el sistema debe definir con precisión qué es una oferta, qué es una acción válida, cuándo existe acuerdo y cómo se evalúa la calidad del resultado. Esta conclusión ha guiado el desarrollo realizado desde la primera entrega.

3. OBJETIVOS CONCRETOS Y METODOLOGÍA DE TRABAJO

3.1.OBJETIVO GENERAL

Diseñar, implementar y evaluar preliminarmente un prototipo software de negociación automatizada en cadena de suministro, con el fin de analizar si un agente basado en LLM puede participar de forma controlada en una negociación de compra-venta sobre precio, cantidad y plazo de entrega, bajo restricciones públicas, guardrails privados, trazabilidad del proceso y métricas de evaluación reproducibles.

3.2.OBJETIVOS ESPECÍFICOS

Definir un modelo de dominio que represente escenarios de negociación, preferencias, guardrails privados, acciones, propuestas, acuerdos y resultados.

Diseñar un protocolo explícito de negociación con acciones como PROPOSE, COUNTER, ACCEPT, REJECT y WALK_AWAY.

Implementar reglas de validación que impidan acuerdos fuera de restricciones públicas o contrarios a los límites privados de aceptación de cada agente.

Desarrollar un motor de negociación por turnos con trazabilidad del estado, registro de propuestas y control del cierre de acuerdos.

Incorporar un proveedor determinista como línea base y un proveedor LLM local mediante Ollama para generar acciones de negociación estructuradas.

Definir métricas básicas para evaluar tasa de acuerdo, validez, viabilidad privada, número de rondas, utilidad de comprador y vendedor, y equilibrio del acuerdo.

Preparar escenarios simulados reproducibles y mecanismos de exportación JSON para facilitar la evaluación experimental del prototipo.

Analizar preliminarmente el comportamiento del proveedor LLM local como posible agente de apoyo en procesos comerciales automatizados, comparándolo con una línea base determinista.

Documentar las decisiones de diseño y mantener el código versionado en un repositorio público.

3.3.METODOLOGÍA DEL TRABAJO

La metodología seguida es de desarrollo iterativo e incremental. En lugar de comenzar por una interfaz visual, el trabajo se ha centrado primero en construir un núcleo de negociación

robusto, testeable y trazable. Esta decisión responde al objetivo académico del TFE: demostrar que el comportamiento de los agentes puede controlarse y evaluarse, antes de abordar capas de presentación.

El desarrollo se ha organizado en iteraciones funcionales. Primero se creó un esqueleto modular del proyecto. Después se refactorizó el dominio para representar acciones explícitas y no acuerdos inferidos. A continuación, se añadieron guardrails privados, estado de negociación, exportación JSON, ejecución por lotes y generación de escenarios simulados. Finalmente, se integró un proveedor local mediante Ollama, manteniendo el mock determinista como baseline.

La metodología de evaluación se basa en escenarios sintéticos reproducibles. Cada escenario define restricciones públicas, preferencias privadas y guardrails de aceptación. El sistema registra todas las acciones y calcula métricas sobre el resultado. En la fase final del TFE se prevé ampliar esta evaluación mediante lotes de simulación que comparen el proveedor determinista y el proveedor LLM local.

Como nota metodológica, durante el desarrollo se han utilizado herramientas de apoyo basadas en IA para acelerar tareas de implementación y revisión, siempre bajo supervisión del autor. Las decisiones de diseño, los cambios funcionales y la evolución del código se documentan mediante control de versiones, pruebas unitarias y un registro de decisiones. Esta información se mantendrá visible para facilitar la trazabilidad y la revisión académica del trabajo.

Tabla 1. Variables negociadas en el prototipo.

Variable	Descripción	Criterio de validez
Precio unitario	Precio por unidad del producto negociado.	Debe ser numérico, positivo y estar dentro del rango público del escenario.
Cantidad	Número de unidades solicitadas/ofrecidas.	Debe ser un entero positivo y estar dentro de los límites del escenario.
Plazo de entrega	Fecha comprometida para la entrega.	Debe situarse entre la fecha mínima y máxima del escenario y respetar guardrails privados al aceptar.

4. DESARROLLO ESPECÍFICO DE LA CONTRIBUCIÓN

4.1.ALCANCE FUNCIONAL DEL PROTOTIPO

La contribución principal desarrollada hasta esta segunda entrega es un prototipo software modular que permite ejecutar negociaciones bilaterales en un entorno controlado. El sistema no pretende optimizar de forma global toda una cadena de suministro, sino estudiar una parte concreta del proceso: la negociación automática de condiciones básicas entre comprador y vendedor. Esta elección permite aproximarse, de forma acotada, a la cuestión de si los agentes basados en LLM pueden actuar en el futuro como apoyo a vendedores, comerciales o teleoperadores comerciales, siempre dentro de mecanismos externos de validación y supervisión.

El alcance actual incluye: generación de escenarios simulados, representación de preferencias y guardrails, motor de negociación por turnos, validación de acciones, cierre explícito de acuerdos, cálculo de métricas, exportación de resultados, ejecución por lotes e integración inicial con un proveedor LLM local mediante Ollama. Quedan fuera de esta entrega la interfaz gráfica final, la evaluación experimental completa y la discusión final de resultados.

4.2.ARQUITECTURA SOFTWARE

La arquitectura se ha diseñado separando responsabilidades. El módulo de modelos contiene las estructuras del dominio; el validador concentra las reglas de consistencia; el motor ejecuta la negociación; el módulo de métricas evalúa los resultados; los módulos de escenarios generan casos sintéticos; y el paquete de proveedores LLM encapsula la interacción con el mock o con Ollama.

Tabla 2. Componentes principales del prototipo.

Componente	Responsabilidad principal
negotiation.models	Define escenarios, acciones, propuestas, acuerdos, estados y resultados.
negotiation.validator	Valida términos, acciones, aceptación de propuestas y acuerdos.
negotiation.engine	Ejecuta la negociación por turnos y mantiene trazabilidad del proceso.

negotiation.metrics	Calcula métricas de validez, utilidad, viabilidad y equilibrio.
negotiation.exporter	Exporta resultados completos a estructuras JSON.
scenarios.generator	Crea escenarios simulados reproducibles.
scenarios.batch	Ejecuta lotes de negociaciones y calcula resúmenes agregados.
experiments.runner	Prepara experimentos reproducibles y exportables.
llm.provider	Implementa el proveedor mock determinista.
llm.ollama_provider	Integra un proveedor LLM local a través de Ollama.
llm.action_parser	Convierte respuestas estructuradas del LLM en acciones del dominio.

Fuente: elaboración propia.

4.3.PROTOCOLO DE NEGOCIACIÓN

El protocolo se basa en acciones explícitas. Esta decisión corrige una debilidad del planteamiento inicial: no basta con que dos ofertas se solapen para considerar que existe acuerdo. En el prototipo actual, cada turno contiene una acción concreta y validable.

Tabla 3. Acciones del protocolo de negociación.

Acción	Significado operativo	Condición principal
PROPOSE	Inicio de una propuesta nueva.	Requiere términos de oferta y no debe apuntar a propuesta previa.

COUNTER	Respuesta directa a una propuesta anterior.	Requiere términos de oferta y referencia a una propuesta válida de la contraparte.
ACCEPT	Aceptación explícita de una propuesta.	Debe apuntar a la última propuesta válida de la contraparte, no rechazada y compatible con guardrails.
REJECT	Rechazo específico de una propuesta.	Requiere referencia a una propuesta concreta y la vuelve no aceptable posteriormente.
WALK_AWAY	Salida de la negociación sin acuerdo.	No requiere términos ni referencia a propuesta.

Fuente: elaboración propia.

4.4.GUARDRAILS, VALIDACIÓN Y TRAZABILIDAD

El sistema incorpora restricciones públicas y guardrails privados. Las restricciones públicas definen el espacio legal de negociación, conocido por ambos agentes. Los guardrails privados representan límites internos de aceptación de cada parte. Por ejemplo, el comprador dispone de un precio máximo aceptable, una cantidad mínima y una fecha límite; el vendedor dispone de precio mínimo, cantidad mínima y fecha mínima de entrega compatible con su capacidad.

Este diseño también permite responder a riesgos como la manipulación entre agentes: aunque un modelo intente influir en otro mediante el contexto, el sistema solo acepta acciones estructuradas que superan las reglas externas de validación.

La validación se aplica en varios niveles: términos de oferta, consistencia de acciones, referencias a propuestas, aceptación de guardrails privados y validez del acuerdo. Si una acción incumple estas reglas, el motor no intenta reinterpretarla ni repararla automáticamente, sino que detiene la negociación como salida inválida. Esta decisión favorece la trazabilidad y evita que el sistema oculte errores del proveedor LLM.

Cada turno registra el estado posterior de la negociación: últimas propuestas válidas por agente, propuestas activas, rechazadas, aceptadas, propuesta activa principal y motivo del

último cambio de estado. Esta trazabilidad permite reconstruir el proceso de negociación y analizar por qué se alcanzó o no un acuerdo.

4.5. INTEGRACIÓN LOCAL CON LLM MEDIANTE OLLAMA

La integración con modelos de lenguaje se ha realizado mediante un proveedor local basado en Ollama. Esta opción permite ejecutar modelos descargados en la máquina del autor, reduciendo dependencia de servicios externos y facilitando pruebas reproducibles. Actualmente, el modelo principal de pruebas locales es gemma4:26b, mientras que qwen3.5:27b mostró timeouts tempranos en este entorno y no se toma por ahora como modelo principal.

El proveedor Ollama solicita al modelo una salida JSON estructurada con `action_type`, `target_offer_id`, `offer_terms` y `rationale`. El parser convierte esta salida en una acción del dominio. Si la respuesta no puede interpretarse, si contiene un `rationale` excesivamente largo o si el proveedor falla, se genera una acción inválida controlada que el motor rechaza. De este modo, el LLM no decide la validez de la negociación: solo propone acciones candidatas.

Tras las primeras pruebas, se redujo la longitud del prompt, se limitó el historial enviado al modelo y se endureció el contrato de salida. Esta modificación responde a un problema observado: algunos modelos pueden generar razonamientos internos o valores inválidos, incluso cuando el formato general parece correcto. La arquitectura actual permite detectar y bloquear estos casos.

5. CÓDIGO FUENTE Y DATOS ANALIZADOS

5.1.REPOSITORIO Y ORGANIZACIÓN DEL CÓDIGO

El código fuente se encuentra versionado en un repositorio público de GitHub: <https://github.com/santimc98/tfg-negociacion-multiagente-llm>. El repositorio se ha organizado de forma modular para facilitar revisión, pruebas y evolución del prototipo. Cada iteración relevante se ha registrado mediante commits descriptivos y un archivo de decisiones de diseño.

La estructura actual del repositorio incluye los paquetes `src/negotiation`, `src/llm`, `src/scenarios` y `src/experiments`, además de tests unitarios. También se mantiene `docs/decision_log.md` como registro de decisiones técnicas y conceptuales adoptadas durante el desarrollo.

5.2.ESCENARIOS SIMULADOS

En esta etapa no se emplean datos reales de empresas. La decisión metodológica es trabajar con escenarios sintéticos porque permiten controlar variables, reproducir experimentos y evitar problemas de confidencialidad. Cada escenario contiene rangos públicos, preferencias privadas y guardrails. Además, el sistema incluye una función de generación de múltiples escenarios con semilla para reproducibilidad.

Los escenarios simulados permiten evaluar el sistema en condiciones variadas sin depender todavía de una fuente externa. En la fase final del TFE, estos escenarios podrán ampliarse con distintos niveles de conflicto entre comprador y vendedor, variaciones en precios, cantidades y plazos, y comparación entre el proveedor mock y el proveedor LLM local.

5.3.PRUEBAS Y RESULTADOS PRELIMINARES

La verificación técnica se ha apoyado en pruebas unitarias con `unittest`. En el estado actual del desarrollo, la suite de pruebas incluye 54 casos y se ejecuta correctamente. Estas pruebas cubren parsing de acciones, validación de términos, semántica del protocolo, guardrails privados, exportación JSON, generación de escenarios, ejecución por lotes y proveedor Ollama mediante respuestas simuladas.

Tabla 4. Métricas previstas para la evaluación.

Métrica	Interpretación
<code>agreement_reached</code>	Indica si la negociación terminó con acuerdo.

valid_agreement	Comprueba si el acuerdo cumple las restricciones públicas.
private_feasibility_buyer	Indica si el acuerdo respeta los guardrails privados del comprador.
private_feasibility_seller	Indica si el acuerdo respeta los guardrails privados del vendedor.
rounds_used	Número de rondas necesarias hasta cierre o parada.
buyer_utility	Utilidad estimada para el comprador según preferencias y rangos.
seller_utility	Utilidad estimada para el vendedor según preferencias y rangos.
agreement_balance_gap	Diferencia absoluta entre utilidad del comprador y del vendedor.

Fuente: elaboración propia.

También se han realizado pruebas manuales preliminares con el proveedor local. Con gemma4:26b se han observado negociaciones con varios turnos válidos, incluyendo propuestas y contraofertas coherentes, aunque en algunos casos el modelo generó términos inválidos. En esos casos, el motor rechazó la salida y detuvo la negociación como invalid_provider_output. Con qwen3.5:27b se observaron timeouts tempranos en el entorno local utilizado. Estos resultados son preliminares y no deben interpretarse como evaluación final.

Tabla 5. Pruebas preliminares realizadas.

Prueba	Resultado observado	Interpretación
Proveedor mock	Ejecución estable y determinista.	Sirve como baseline para comprobar el motor y las métricas.

Ollama con gemma4:26b	Generó varios turnos válidos, aunque también algunas salidas inválidas.	Demuestra integración real con LLM y utilidad de los guardrails.
Ollama con qwen3.5:27b	Timeouts tempranos en el entorno local.	No se toma por ahora como modelo principal de evaluación.
Pruebas unitarias	54 tests ejecutados correctamente.	Aporta confianza en protocolo, validación y exportación.

Fuente: elaboración propia.

6. CONCLUSIONES Y TRABAJO FUTURO

6.1.CONCLUSIONES PROVISIONALES

La segunda entrega muestra un avance funcional relevante respecto a la primera. El trabajo ya no se limita a una idea de agentes negociando, sino que dispone de un núcleo software que define acciones, valida propuestas, aplica guardrails, registra el estado de la negociación y permite integrar un proveedor LLM local de forma controlada. Con ello, la finalidad del TFE queda mejor concretada: evaluar la viabilidad preliminar de un LLM como agente negociador en una compra-venta simulada, no delegar directamente decisiones comerciales reales en el modelo.

Una conclusión provisional importante es que la integración de LLMs en negociación automática requiere una arquitectura supervisada. Las pruebas iniciales muestran que el modelo puede generar acciones razonables, pero también respuestas inválidas. Por tanto, el diseño adoptado, en el que el motor y el validador son la autoridad final, resulta adecuado y defendible.

Otra conclusión preliminar es que los escenarios simulados son una estrategia apropiada para esta fase del TFE. Permiten evaluar comportamiento, reproducir experimentos y controlar el grado de conflicto sin depender de datos reales.

6.2.LIMITACIONES Y PROSPECTIVA

El prototipo todavía presenta limitaciones. La evaluación experimental sistemática no está completada; la interfaz de usuario no se ha desarrollado; el proveedor LLM local requiere más pruebas de estabilidad; y los escenarios simulados son aún simplificados. Además, las métricas actuales ofrecen una aproximación básica de utilidad y equilibrio, pero podrían ampliarse con análisis de eficiencia, robustez y sensibilidad. Tampoco puede concluirse todavía que un LLM esté preparado para sustituir a un vendedor o teleoperador comercial real, sino únicamente que puede estudiarse su comportamiento en un entorno controlado y validado.

Como trabajo inmediato se plantea ejecutar experimentos comparativos entre el proveedor mock y gemma4:26b en lotes reproducibles, analizar la frecuencia de acuerdos, identificar errores recurrentes del proveedor LLM y valorar si conviene incorporar reintentos controlados o una estrategia de fallback. En fases posteriores también se podrá estudiar una interfaz mínima para visualizar la conversación, el estado y las métricas de cada negociación.

7. REFERENCIAS BIBLIOGRÁFICAS

- Bianchi, F., Chia, P. J., Yüksekönül, M., Tagliabue, J., Jurafsky, D., & Zou, J. (2024). How well can LLMs negotiate? NegotiationArena platform and analysis. Proceedings of the 41st International Conference on Machine Learning.
- Davidson, T. R., Veselovsky, V., Kosinski, M., & West, R. (2024). Evaluating language model agency through negotiations. International Conference on Learning Representations.
- Fang, F., & Wong, T. N. (2010). Applying hybrid case-based reasoning in agent-based negotiations for supply chain management. *Expert Systems with Applications*, 37(12), 8322-8332. <https://doi.org/10.1016/j.eswa.2010.05.042>
- Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence.
- Guttman, R. H., Moukas, A. G., & Maes, P. (1998). Agent-mediated electronic commerce: A survey. *The Knowledge Engineering Review*, 13(2), 147-159. <https://doi.org/10.1017/S0269888998002082>
- Jennings, N. R., Faratin, P., Lomuscio, A. R., Parsons, S., Sierra, C., & Wooldridge, M. (2001). Automated negotiation: Prospects, methods and challenges. *Group Decision and Negotiation*, 10, 199-215. <https://doi.org/10.1023/A:1008746126376>
- Wang, G., Wong, T. N., & Wang, X. (2013). An ontology based approach to organize multi-agent assisted supply chain negotiations. *Computers & Industrial Engineering*, 65(1), 2-15. <https://doi.org/10.1016/j.cie.2011.11.014>
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J.-R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18, 186345.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations.

8. ANEXOS

En esta segunda entrega no se incorporan anexos extensos. Para la entrega final se prevé incluir, si procede, ejemplos completos de salida JSON, capturas de ejecuciones experimentales y fragmentos representativos de configuración del prototipo.